

# Especialização em Ciência de Dados

2026.2

## Introdução à Programação para Ciência de Dados

Prof. Ricardo Sovat  
sovat@ifsp.edu.br



**INSTITUTO FEDERAL**  
São Paulo  
Câmpus Campinas

## NumPy (3)

## Agregações

O NumPy possui um conjunto de funções agregadoras para os arrays (funções que recebam o vetor inteiro como parâmetro e o percorram para o cálculo).

Isto é especialmente útil para cálculo de estatísticas sobre o conjunto de valores expresso pelo vetor

As mais comuns são a média e o desvio-padrão, mas também existem outras como soma, produto, mediana, valores mínimo e máximo, por exemplo

## Agregações

```
np.sum(Lista)
```

```
np.float64(48.36116195455385)
```

```
array_grande = rng.random(1000000)
```

```
print(array_grande.min())
```

```
print(array_grande.max())
```

```
print(array_grande.sum())
```

```
7.853658696710397e-07
```

```
0.9999979953129792
```

```
499705.6750712082
```

# Broadcasting

## Regras

1. Se dois arrays forem de **dimensões diferentes**, o formato do array com o menor número de dimensões é preenchido com 1s (uns) à esquerda
2. Se o **formato** dos dois arrays não corresponder em nenhuma de suas dimensões, o array com o formato igual a 1 em uma de suas dimensões será estendido para combinar com o outro
3. Se os **tamanhos** não corresponderem em qualquer dimensão e nenhum for igual a 1, a operação resultará em um erro

## Broadcasting

array + array

```
a = np.array([0, 1, 2])  
b = np.array([5, 5, 5])  
a + b
```

```
array([5, 6, 7])
```

```
a + 5
```

```
array([5, 6, 7])
```

# Broadcasting

Matriz + array

```
Matriz = np.ones((3,3))
```

```
Matriz
```

```
array([[1., 1., 1.],  
       [1., 1., 1.],  
       [1., 1., 1.]])
```

Matriz + a

```
array([[1., 2., 3.],  
       [1., 2., 3.],  
       [1., 2., 3.]])
```

# Broadcasting

array linha  
+  
array coluna

```
a = np.arange(3)
b = np.arange(3)[: , np.newaxis]
print (a)
print (b)
```

```
[0 1 2]
[[0]
 [1]
 [2]]
```

a + b

```
array([[0, 1, 2],
       [1, 2, 3],
       [2, 3, 4]])
```



# Broadcasting

$$\begin{bmatrix} 0 & 1 & 2 \end{bmatrix} + \begin{bmatrix} 5 & 5 & 5 \end{bmatrix} = \begin{bmatrix} 5 & 6 & 7 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 2 & 3 \\ 2 & 3 & 4 \end{bmatrix}$$

Visualização do broadcasting  
no NumPy

# Máscaras

Existe no NumPy a possibilidade de serem criados padrões mais poderosos para cálculos: arrays booleanos como “máscaras” para selecionar subconjuntos de dados específicos.

x

```
array([[1, 9, 2, 9],  
       [7, 8, 0, 9],  
       [4, 8, 5, 3]])
```



x < 5

```
... array([[ True, False,  True, False],  
          [False, False,  True, False],  
          [ True, False, False,  True]])
```

# Indexação Avançada

A indexação avançada (*fancy indexing*) é simples porém muito prática: significa passar um índice para acessar múltiplos elementos do array de uma só vez:

```
x = rng.integers(100, size=10)
print (x)
```

```
[22 91 47  1 84 16 74 82 70 70]
```

```
# Podemos acessar três elementos diferentes da seguinte forma:
```

```
[x[3],x[7],x[2]]
```

```
[np.int64(1), np.int64(82), np.int64(47)]
```

```
# mas também podemos passar uma lista de índices e obter o mesmo resultado:
```

```
indices = [3, 7, 4]
```

```
x[indices]
```

## Indexação Avançada

```
indices = np.array([[3,7],[4,5]])  
x[indices]
```

```
[22 91 47  1 84 16 74 82 70 70]
```

```
array([[ 1, 82],  
       [84, 16]])
```

```
# funciona também em múltiplas dimensões  
x = np.arange(12).reshape((3,4))  
x
```

```
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

```
linhas = np.array([0,1,2])  
colunas = np.array([2,1,3])  
x[linhas,colunas]
```

```
array([ 2,  5, 11])
```

# Indexação Avançada

Funciona também associado aos tópicos anteriores:

```
#indexação simples junto com a avançada  
x[2, [2,0,1]]
```

```
array([10,  8,  9])
```

```
#indexação avançada com slicing  
x[1:,[2,0,1]]
```

```
array([[ 6,  4,  5],  
       [10,  8,  9]])
```

```
#indexação avançada e máscaras  
mascara = np.array([True, False, True, False])  
x[linhas[:,np.newaxis], mascara]
```

```
array([[ 0,  2],  
       [ 4,  6],  
       [ 8, 10]])
```

## Ordenação

Função *sort* : aceita uma lista e retorna uma versão ordenada dessa mesma linha:

```
# NO NUMPY
x = np.array([2, 1, 4, 3, 5])
np.sort(x) #análoga à função sorted do Python, retorna uma cópia

array([1, 2, 3, 4, 5])
```

Método *sort* : ordena a lista original

```
x.sort() # análogo ao método sort do Python: altera o original
print(x)

[1 2 3 4 5]
```

## Arrays Estruturados

Às vezes é conveniente introduzirmos alguma estrutura nos vetores, ou seja, não usar um array homogêneo

```
# Não há relacionamento explícito entre os seguintes vetores:
```

```
nome = ['Alice', 'Bob', 'Carlos', 'Dora']
```

```
idade = [25, 45, 37, 19]
```

```
peso = [55.0, 85.5, 68.0, 61.5]
```

```
dados = np.zeros(4, dtype=[('nome', 'U10'), ('idade', 'i4'), ('peso', 'f8')])  
print(dados.dtype)
```

```
[('nome', '<U10'), ('idade', '<i4'), ('peso', '<f8')]
```

## Arrays Estruturados

```
dados['nome']= nome  
dados['idade']= idade  
dados['peso']= peso  
print(dados)
```

```
[('Alice', 25, 55. ) ('Bob', 45, 85.5) ('Carlos', 37, 68. )  
 ('Dora', 19, 61.5)]
```



# Podemos obter, por exemplo, todos os nomes

```
dados['nome']
```

```
... array(['Alice', 'Bob', 'Carlos', 'Dora'], dtype='<U10')
```



# Arrays Estruturados

| Caractere | Descrição                                          | Exemplo                        |
|-----------|----------------------------------------------------|--------------------------------|
| '?'       | Booleano (bool)                                    | <code>np.dtype('?')</code>     |
| 'b'       | Byte — inteiro com sinal de 8 bits                 | <code>np.dtype('b')</code>     |
| 'B'       | Byte sem sinal — inteiro de 8 bits                 | <code>np.dtype('B')</code>     |
| 'h'       | Inteiro curto (short)                              | <code>np.dtype('h')</code>     |
| 'H'       | Inteiro curto sem sinal (unsigned short)           | <code>np.dtype('H')</code>     |
| 'i'       | Inteiro (int)                                      | <code>np.dtype('i')</code>     |
| 'I'       | Inteiro sem sinal (unsigned int)                   | <code>np.dtype('I')</code>     |
| 'l'       | Inteiro longo (long)                               | <code>np.dtype('l')</code>     |
| 'L'       | Inteiro longo sem sinal (unsigned long)            | <code>np.dtype('L')</code>     |
| 'q'       | Inteiro longo longo (long long)                    | <code>np.dtype('q')</code>     |
| 'Q'       | Inteiro longo longo sem sinal (unsigned long long) | <code>np.dtype('Q')</code>     |
| 'e'       | Ponto flutuante de 16 bits (float16)               | <code>np.dtype('e')</code>     |
| 'f'       | Ponto flutuante de 32 bits (float32)               | <code>np.dtype('f')</code>     |
| 'd'       | Ponto flutuante de 64 bits (float64)               | <code>np.dtype('d')</code>     |
| 'g'       | Ponto flutuante de precisão estendida (longdouble) | <code>np.dtype('g')</code>     |
| 'F'       | Complexo de 64 bits (complex64)                    | <code>np.dtype('F')</code>     |
| 'D'       | Complexo de 128 bits (complex128)                  | <code>np.dtype('D')</code>     |
| 'G'       | Complexo de precisão estendida (clongdouble)       | <code>np.dtype('G')</code>     |
| 'S'       | String de bytes de tamanho fixo                    | <code>np.dtype('S10')</code>   |
| 'U'       | String Unicode de tamanho fixo                     | <code>np.dtype('U10')</code>   |
| 'V'       | Dados brutos (void) de tamanho fixo                | <code>np.dtype('V8')</code>    |
| 'O'       | Objeto Python                                      | <code>np.dtype('O')</code>     |
| 'M'       | Data e hora (datetime64)                           | <code>np.dtype('M8[D]')</code> |
| 'm'       | Intervalo de tempo (timedelta64)                   | <code>np.dtype('m8[D]')</code> |